

Learning C Through Small Projects

This document records my progress learning the C programming language through a structured series of small, hands-on projects.

Each lesson introduces one major concept and builds on the previous exercises.

Learning roadmap

Lesson	Main concept	Project / focus	Status
01	Compilation, Make, and <code>main()</code>	Hello World	Completed
02	<code>argc</code> , <code>argv</code> , and loops	Command-line arguments	Completed
03	Variables, arithmetic, and conditionals	Simple calculator	Completed
04	Loops	Multiplication table	Completed
05	Functions	Refactor calculator	Completed
06	Header files	Split project into modules	Completed
07	Arrays and indexing	Bounds, loops, element count	In progress
08	Object sizes and <code>sizeof</code>	Array sizing and object representation	Planned
09	Pointer fundamentals	Addresses, <code>&</code> , and <code>*</code>	Planned
10	Arrays vs. pointers	Decay, function parameters, and <code>sizeof</code> differences	Planned
11	Strings	Character arrays, <code>'\0'</code> , <code>strlen()</code> , and tiny echo	Planned
12	File I/O	Count lines in a file	Planned
13	Structs	Contact catalog	Planned
14	Dynamic memory	Allocation and lifetime	Planned
15	Memory addresses and offsets	Hex addresses and byte offsets	Planned
16	Pointer arithmetic	Manual traversal	Planned
17	Raw memory and buffers	Byte-buffer inspection	Planned
18	Memory layout and padding	Alignment and struct padding	Planned
19	Memory safety	Bounds, lifetime, and undefined behavior	Planned
20	Binary file I/O	Read and inspect raw bytes	Planned
21	Combined fundamentals	Tiny Unix utility	Planned

The later memory-focused lessons build progressively from addresses and pointer fundamentals to offsets, pointer arithmetic, raw byte buffers, alignment, buffer layouts, allocation lifetime, and memory safety. These lessons will explicitly distinguish defined behavior from implementation-defined, unspecified, and undefined behavior.

Completed lessons

Lesson 1: Hello World

This lesson introduced the basic structure of a C program, including `#include`, the `main()` function, `printf()`, compilation with GCC, and automated builds using Make.

Lesson 2: Command-line arguments

This lesson introduced `argc` and `argv`, which allow a C program to receive arguments from the command line. It also introduced program exit codes and basic input validation.

Lesson 3: Simple calculator

This lesson introduced variables, numeric types, arithmetic operators, integer and floating-point division, and protection against division by zero.

Lesson 4: Loops and control flow

This lesson introduced:

- for loops
- while loops
- if-else if-else conditional branching
- break to exit a loop
- continue to skip the remainder of an iteration
- return to exit the current function

The exercises demonstrated that statements execute sequentially. Control-flow statements only affect code that has not yet executed.

For example:

```
if (i == 5) {  
    continue;  
}  
  
printf("%d\n", i);
```

Equivalent Python exercises were created to compare universal programming concepts with language-specific syntax.

Lesson 5: Functions

This lesson introduced defining and calling functions, passing parameters, and using return values. The calculator was refactored into focused helper functions for addition, subtraction, multiplication, integer division, remainder, and decimal division while retaining the division-by-zero guard in `main()`. A Python comparison exercise reinforced that decomposing a program into functions is a universal design concept, even though C and Python express types, division, and program structure differently.

Lesson 6: Header Files

This lesson introduced the separation of function declarations from function definitions. Public declarations were placed in `calculator.h`, function definitions were moved to `calculator.c`, and `main()` remained in `calculator-main.c`. Include guards prevented the header contents from being processed more than once within a translation unit.

The calculator was built through separate compilation: each source file was compiled into its own object file, and the linker combined those object files into the final executable. The Makefile tracked source and header dependencies so that only affected files were recompiled. Including `calculator.h` from `calculator.c` also allowed the compiler to verify that each function definition matched its advertised declaration.

Translation unit	Knows about	Produces
<code>calculator-main.c</code> plus its headers	Function declarations and <code>main()</code>	<code>calculator-main.o</code>
<code>calculator.c</code> plus its header	Declarations and function definitions	<code>calculator.o</code>

Doing hands-on exercises and learning to reason through Make dependency behavior, compilation/linking stages, and runtime program logic. Also covered topics such as:

- `-c` means: compile and assemble, but stop before linking.
- `-o` filename means: name the output file filename.
- `.o` is the conventional extension for an object file.
- `.h` is the conventional extension for a header file.

Comparative exercises

Selected C exercises are recreated in other languages when the comparison strengthens understanding.

Python provides comparison with a higher-level general-purpose language. PowerShell will provide comparison with scripting, structured objects, pipelines, and Windows automation.

C remains the primary learning track.

Current position

Arrays and Indexing